

Facundo Gael Piñeiro González

A01666626

1 de Junio de 2026

Evidencia/Reflexión: STC0205 Elaboración de pruebas de software

¿Qué es una prueba de software?

Una prueba de software es una verificación que permite comprobar si un sistema produce el resultado esperado para una determinada entrada. Su objetivo es detectar defectos antes de que el sistema sea utilizado por los usuarios finales. Una prueba se considera exitosa cuando el resultado obtenido coincide con el resultado esperado.

Error, defecto y falla

Aunque suelen utilizarse como sinónimos, representan conceptos distintos:

- **Error:** Equivocación humana durante el análisis, diseño o programación.
- **Defecto:** Problema presente dentro del código o sistema como consecuencia de un error.
- **Falla:** Comportamiento incorrecto observado por el usuario cuando el defecto es ejecutado.

Comprender esta diferencia ayuda a identificar mejor el origen de los problemas encontrados durante las pruebas.

Caso de prueba

Un caso de prueba es un conjunto de condiciones utilizadas para verificar una funcionalidad específica del sistema. Generalmente incluye:

- Identificador.
- Objetivo.
- Precondiciones.
- Datos de entrada.
- Pasos de ejecución.
- Resultado esperado.

Los casos de prueba permiten documentar de manera formal qué funcionalidad se está validando y facilitan la repetición de las pruebas.

Verificación y validación

Las pruebas de software pueden enfocarse en dos perspectivas:

Verificación

- Comprueba si el sistema fue construido correctamente.

- Se enfoca en el funcionamiento interno del software.
- Incluye pruebas unitarias, de integración y de sistema.

Validación

- Comprueba si el sistema satisface las necesidades del usuario.
- Se enfoca en el uso real del producto.
- Incluye pruebas de aceptación y pruebas de usabilidad.

Tipos de pruebas

Existen distintos tipos de pruebas según el objetivo que se desea alcanzar:

- **Pruebas unitarias:** Validan funciones o componentes individuales.
- **Pruebas de integración:** Verifican la interacción entre módulos.
- **Pruebas de sistema:** Evalúan el funcionamiento completo del sistema.
- **Pruebas de aceptación:** Comprueban el cumplimiento de los requerimientos del cliente.
- **Pruebas de usabilidad:** Analizan la experiencia de los usuarios al utilizar el sistema.

Generación de casos de prueba a partir de casos de uso

Los casos de uso pueden utilizarse como base para generar casos de prueba. Cada flujo principal y flujo alternativo descrito en un caso de uso representa un escenario que debe ser validado.

El proceso general consiste en:

1. Identificar los escenarios del caso de uso.
2. Definir casos de prueba para cada escenario.
3. Determinar los datos de entrada necesarios.
4. Establecer los resultados esperados.

Esta metodología permite asegurar que las pruebas cubran tanto los escenarios exitosos como los escenarios de error.

Importancia de las pruebas en el proyecto

En el proyecto desarrollado para el socio formador, los casos de prueba permitieron validar que los requerimientos funcionales fueran implementados correctamente. Los ejemplos presentados para el RF-02 muestran tanto un escenario exitoso de vinculación cliente-operación como un escenario alternativo donde el cliente no existe. De esta manera se verifica que el sistema responda adecuadamente tanto en condiciones normales como en situaciones excepcionales.

Ejemplos de casos de pruebas realizados para el proyecto con el socioformador:

Caso de Prueba CU-02.1 - Vinculación Cliente-Operación RF-02

Requerimiento: RF-02

Caso de uso relacionado: CU-02

Nombre: Vinculación Cliente-Operación

Campo	Descripción
Objetivo	Permite asociar una operación registrada con un cliente existente en el sistema.
Actor	Usuario (Empleado / Oficial de Cumplimiento)
Precondiciones	El usuario está autenticado Existe al menos un cliente registrado en el sistema. Existe una operación registrada pendiente de vinculación.
Datos de entrada	ID del cliente, RFC y datos de la operación.
Pasos	1. El usuario selecciona la opción “Registrar operación”. 2. El usuario ingresa los datos de la operación. 3. El sistema solicita la identificación del cliente. 4. El sistema proporciona los datos del cliente (ID, RFC, etc.). 5. El sistema valida la existencia del cliente.

	6. El sistema vincula la operación con el cliente. 7. El sistema guarda la relación. 8. El sistema confirma la vinculación.
Resultado esperado	La operación queda vinculada correctamente con el cliente registrado.
Resultado obtenido	Pendiente de ejecución.
Estado	Pendiente / Aprobado / Fallido

Caso de Prueba CP-02.2 - Cliente no encontrado

Requerimiento: RF-02

Caso de uso relacionado: CU-02

Nombre: Validación de cliente inexistente en vinculación

Campo	Descripción
Objetivo	Verificar que el sistema detecte cuando el cliente no existe y no permita realizar la vinculación.
Actor	Usuario (Empleado / Oficial de Cumplimiento)
Precondiciones	El usuario está autenticado Existe una operación registrada pendiente de vinculación.
Datos de entrada	ID del cliente, RFC y datos de la operación.
Pasos	1. El usuario selecciona la opción “Registrar operación”. 2. El usuario ingresa los datos de la operación. 3. El sistema solicita la identificación del cliente. 4. El usuario ingresa un ID o RFC inexistente. 5. El sistema valida la existencia del cliente. 6. El sistema detecta que el cliente no existe. 7. El sistema muestra el mensaje “Cliente no encontrado”.
Resultado esperado	El sistema no realiza la vinculación y finaliza el proceso mostrando el error correspondiente.
Resultado obtenido	Pendiente de ejecución.
Estado	

	Pendiente / Aprobado / Fallido
--	-----------------------------------

Evidencia 2do Intento:

Reflexión y Correcciones con respecto a la anterior evidencia:

En la entrega anterior no demostré un análisis de competencia, ya que utilicé IA para su elaboración. Para esta evidencia, repasaré los temas de una manera más informal y concisa. Además, incluiré evidencias y una reflexión con respecto al proyecto, que era lo que faltaba en la entrega pasada.

Resumen de los conceptos claves de pruebas de software.

Entonces que son?

Las pruebas de software son un proceso de verificación que sirve principalmente para comprobar que el sistema produzca el resultado esperado. Una prueba es exitosa cuando el resultado obtenido coincide con lo que se esperaba.

Error - errores de programación (escritura de código o diseño).

Defecto - Problemas que quedan en el código por culpa del error.

Falla - Lo que ve un usuario cuando el defecto es ejecutado

En otras palabras, son conceptos que van de la mano como parte del ciclo de vida de un bug: primero ocurre un error interno en el código que genera un defecto, y este finalmente se hace visible al usuario como una falla.

Verificación - Responde a la pregunta ¿Construimos el sistema correctamente?

Validación - Responde a la pregunta ¿Construimos el sistema correcto?

Tipos de pruebas utilizadas en clase:

Unitaria - Verifica que un modulo funcione correctamente (aislado).

Integración - Varios - Verifica que dos o más modulos trabajen correctamente (dos o más).

Casos de prueba:

Son basicamente un formato para describir pruebas. Lo que lo caracteriza es:

ID -> Objetivo -> Precondiciones - > Entrada -> Resultado

Aplicaciones de pruebas en el proyecto:

Aplicamos casos de prueba antes de codificar para definir los resultados esperados de cada funcionalidad de la página. Conocer el flujo de estos casos nos ayudó a visualizar cómo se vería el sistema y qué lo compondría antes de comenzar a programar (Imágenes de casos de prueba de la evidencia 1).

Al momento de conectar la base de datos en los módulos del proyecto por ejemplo la dashboard, realice tests para las áreas necesarias. De esta manera, cuando la información no se mostraba correctamente pero tampoco lanzaba un error en consola, podía identificar con mayor detalle en qué función ocurrió el problema mediante el comando npm test.

Ejemplo en [dashboard.test.js](#):

```
// Tests
describe("Dashboard Controller", () => {

  beforeEach(() => {
    jest.clearAllMocks();
    dashboardModel.getResumen.mockResolvedValue(mockResumen);
    dashboardModel.getPersonasBloqueadas.mockResolvedValue(mockBloqueados);
    dashboardModel.getAlertasRecientes.mockResolvedValue(mockAlertas);
    dashboardModel.getOperacionesRecientes.mockResolvedValue(mockOperaciones);
  });

  // Render principal
  test("renderiza dashboard con todos los datos", async () => {
    const { req, res } = mockReqRes();
    await dashboardController.renderDashboard(req, res);

    expect(res.render).toHaveBeenCalledWith("dashboard", expect.objectContaining({
      resumen: mockResumen,
      bloqueados: mockBloqueados,
      alertas: mockAlertas,
      operaciones: mockOperaciones,
    }));
  });

  test("pasa tiempoRelativo como función al render", async () => {
    const { req, res } = mockReqRes();
    await dashboardController.renderDashboard(req, res);

    const args = res.render.mock.calls[0][1];
    expect(typeof args.tiempoRelativo).toBe("function");
  });

  // Error de BD
  test("responde 500 si el modelo falla", async () => {
    const { req, res } = mockReqRes();
    dashboardModel.getResumen.mockRejectedValue(new Error("DB caída"));

    await dashboardController.renderDashboard(req, res);

    expect(res.status).toHaveBeenCalledWith(500);
    expect(res.send).toHaveBeenCalledWith("Error al cargar el dashboard");
  });

  // tiempoRelativo
  describe("tiempoRelativo()", () => {
    const { tiempoRelativo } = dashboardController;

    test("segundos recientes", () => {
      const fecha = new Date(Date.now() - 30 * 1000);
      expect(tiempoRelativo(fecha)).toBe("Hace 30 seg");
    });

    test("minutos", () => {
      const fecha = new Date(Date.now() - 10 * 60 * 1000);
      expect(tiempoRelativo(fecha)).toBe("Hace 10 min");
    });

    test("horas", () => {
      const fecha = new Date(Date.now() - 3 * 3600 * 1000);
    });
  });
});
```